

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <fcntl.h>
4  #include <sys/mman.h>
5  #include <unistd.h>
6  #include <semaphore.h>
7  #include <time.h>
8  #include <errno.h>
9
10 #define SHM_NAME "/arche_noah_shm"
11 #define SEM_MUTEX "/arche_noah_mutex"
12 #define SEM_FULL "/arche_noah_full"
13 #define N 10
14
15 struct shared_data
16 {
17     int bookings[N];
18     int count;
19 };
20
21 void sem_wait_intr(sem_t *sem)
22 {
23     while (sem_wait(sem) == -1)
24     {
25         if (errno != EINTR)
26         {
27             perror("sem_wait");
28             exit(1);
29         }
30     }
31 }
32
33 int already_used(struct shared_data *data, int id)
34 {
35     for (int i = 0; i < N; i++)
36     {
37         if (data->bookings[i] == id)
38             return 1;
39     }
40     return 0;
41 }
42
43 int main()
44 {
45     srand(time(NULL) ^ getpid());
46
47     int shm_fd = shm_open(SHM_NAME, O_RDWR | O_CREAT, 0666);
48     ftruncate(shm_fd, sizeof(struct shared_data));
49
50     struct shared_data *data = mmap(NULL, sizeof(*data),
51                                     PROT_READ | PROT_WRITE,
52                                     MAP_SHARED, shm_fd, 0);
53
54     sem_t *mutex = sem_open(SEM_MUTEX, O_CREAT, 0666, 1);
55     sem_t *full = sem_open(SEM_FULL, O_CREAT, 0666, 0);
56
57     if (mutex == SEM_FAILED || full == SEM_FAILED)
58     {
59         perror("sem_open");
60         exit(1);
61     }
62
63     sem_wait_intr(mutex);
64
65     if (data->count >= N)
66     {
67         sem_post(mutex);

```

alles schliessen nicht vergessen

```
68         return 0;
69     }
70
71     int id;
72     do
73     {
74         id = rand() % 10000 + 1;
75     } while (already_used(data, id));
76
77     data->bookings[data->count] = id;
78     data->count++;
79
80     printf("Client %d reserves Space %d\n", id, data->count);
81
82     if (data->count == N)
83     {
84         sem_post(full);
85     }
86
87     sem_post(mutex);
88
89     munmap(data, sizeof(*data));
90     close(shm_fd);
91     auch hier alles schliessen
92     return 0;
93 }
94
```